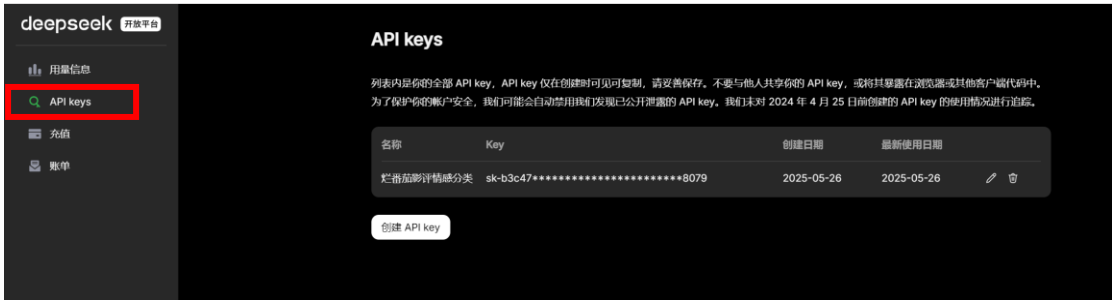
调用闭源模型的 API

以 Deepseek 为例：进入 https://api-docs.deepseek.com/ 查看相关信息，包括价格、调用的代码和注意事项。

然后进入右上角的 DeepSeek 平台创建 API，给 API 命名，要记得复制并提前备份好 Key，因为退出了复制界面后无法查看完整的 API Key。



下载需要的包和数据集。

```
[1] !pip install datasets openai
    !pip install --upgrade datasets
```

```
[2] from datasets import load_dataset

    #下载数据集
    data = load_dataset("rotten_tomatoes")
    data
```

```
README.md: 100% 7.46k/7.46k [00:00<00:00, 581kB/s]
train.parquet: 100% 699k/699k [00:00<00:00, 16.3MB/s]
validation.parquet: 100% 90.0k/90.0k [00:00<00:00, 7.51MB/s]
test.parquet: 100% 92.2k/92.2k [00:00<00:00, 10.3MB/s]
Generating train split: 100% 8530/8530 [00:00<00:00, 133905.52 examples/s]
Generating validation split: 100% 1066/1066 [00:00<00:00, 46379.55 examples/s]
Generating test split: 100% 1066/1066 [00:00<00:00, 61979.35 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 8530
    })
    validation: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
})
```

调用模型的步骤如下：首先创建客户端，输入提前复制好的 API Key。

```
from openai import OpenAI

#创建客户端
client = OpenAI(api_key="sk-b3[redacted]079", base_url="https://api.deepseek.com")
```

定义模型调用函数如下：输入参数有提示语句 prompt，要分析的文档 document 和指定的模型。

```python
def deepseek_generation(prompt,document,model="deepseek-chat"):
    """Generate an output baesd on a prmpt and an input document."""
    messages=[
            {"role": "system",
            "content": "You are a helpful assistant"},
            {"role": "user",
            "content": prompt.replace("[DOCUMENT]",document)}
            #将prompt中出现的字符串“document”替换为document中的字符串，即影评
    ]

    #生成模型的回答
    response = client.chat.completions.create(
        messages=messages,
        model=model,
        stream=False
    )

    print(response.choices[0].message.content)#“0”表示选择返回概率最高的回答
```

```python
#Define a prompt template as a base
prompt = """Predict whether the following document is a positive or negative movie review:

[DOCUMENT]

If it is positive return 1 and if it is negative return 0. Do not give any other answers.
"""

#Predict the target using Deepseek
document = "unpretentious, charming, quirky, original" #朴实、迷人、古怪、原创

#调用模型
deepseek_generation(prompt,document)
```

```
1
```

先给一个文档进行测试“朴实的、迷人的、古怪的和原创的”，返回 1，说明模型认为这条评论是 positive。

```python
#接下来将整个数据集输入到模型当中
import numpy as np
from tqdm import tqdm

#从测试数据中提取document
predictions = [deepseek_generation(prompt,doc) for doc in tqdm(data["test"]["text"])]
```

```
  0%|          | 1/1066 [00:04<1:13:08,  4.12s/it]1
  0%|          | 2/1066 [00:07<1:08:16,  3.85s/it]1
```

最后将整个测试集导入到模型中。

```
[ ]  from sklearn.metrics import classification_report

     def evaluate_performance(y_ture,y_pred):
         """Create and print the classification report"""
         performance = classification_report(
                 y_ture,y_pred,
                 target_names=["Negative Review","Positive Review"]
         )
         print(performance)
```

```
[ ]  #Extract predictions
     y_pred = [int(pred) for pred in predictions]

     #Evaluate performance
     evaluate_performance(data["test"]["label"],y_pred)
```

开源模型

Representation model

运用一个经过了预训练和微调的表示模型，这个模型是用 twitter 的评论数据进行微调的，用这个模型将烂番茄评论数据集分类，判断评论是正面的还是负面的。性能表显示，该模型的性能优异,能够较好地理解文本中附带的情感色彩。

```
%%capture
!pip install datasets transformers sentence-transformers openai
```

```
[ ]  !pip install --upgrade datasets
```

```
from datasets import load_dataset

#下载数据
data=load_dataset("rotten_tomatoes")
```

```
data
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 8530
    })
    validation: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 1066
    })
})
```

```
from transformers import pipeline

#Path to our HF model
model_path = "cardiffnlp/twitter-roberta-base-sentiment-latest"

#下载模型到管道中
pipe = pipeline(
        model = model_path,
        tokenizer = model_path,
        return_all_scores = True,
        device = "cuda:0"
)
```

调用 Transformers 库的 pipeline 接口，这是 Hugging Face 提供的高级 API，可以简化模型调用流程。适用于：文本翻译、问答生成、内容摘要、对话系统等需要文本转换的任务。

```
import numpy as np
from tqdm import tqdm
from transformers.pipelines.pt_utils import KeyDataset

#Run inference
y_pred = []
for output in tqdm(pipe(KeyDataset(data["test"],"text")),total=len(data["test"])):
    #0代表负性，1代表中性，2代表正性
    negative_score = output[0]["score"]
    positive_score = output[2]["score"]
    assignment = np.argmax([negative_score,positive_score])#比较正负分数的大小
    #如果负分数的值更高，那么输出的label是0，反之输出1
    y_pred.append(assignment)
```

```
100%|██████████| 1066/1066 [00:12<00:00, 88.15it/s]
```

```
y_pred[-1]
```

```
np.int64(0)
```

```
from sklearn.metrics import classification_report

def evaluate_performance(y_true,y_pred):
    """Create and print the classification report"""
    performance = classification_report(
            y_true,y_pred,
            target_names = ["Negative Review","Positive Review"]
    )
    print(performance)
```

```
evaluate_performance(data["test"]["label"],y_pred)
```

```
              precision    recall  f1-score   support

Negative Review       0.76      0.88      0.81       533
Positive Review       0.86      0.72      0.78       533

      accuracy                           0.80      1066
     macro avg       0.81      0.80      0.80      1066
  weighted avg       0.81      0.80      0.80      1066
```

Generative model

```
[1]  !pip install datasets transformers sentence-transformers openai
     #安装数据集和transformer，由hugging face提供的包
```

```
[2]  !pip install --upgrade datasets
```

```
from datasets import load_dataset

#下载数据
data = load_dataset("rotten_tomatoes")
data
```

```
        train: Dataset({
            features: ['text', 'label'],
            num_rows: 8530
        })
        validation: Dataset({
            features: ['text', 'label'],
            num_rows: 1066
        })
        test: Dataset({
            features: ['text', 'label'],
            num_rows: 1066
        })
    })
```

```
from transformers import pipeline

#下载模型
pipe = pipeline(
        "text2text-generation",
        model="google/flan-t5-small",
        #可以在hugging face上找到small,large,xxl等不同大小的t5模型
        device="cuda:0"
)
```

```python
#准备数据，给出提示
prompt = "Is the following sentence positive or negative?"
data = data.map(lambda example:{"t5":prompt + example['text']})
#构建新的项“t5”，包含提示和相应的句子评论；随后将新的data传入模型中
data
```

```
Map: 100% 8530/8530 [00:00<00:00, 25339.07 examples/s]
Map: 100% 1066/1066 [00:00<00:00, 13679.99 examples/s]
Map: 100% 1066/1066 [00:00<00:00, 17211.41 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 't5'],
        num_rows: 8530
    })
    validation: Dataset({
        features: ['text', 'label', 't5'],
        num_rows: 1066
    })
    test: Dataset({
        features: ['text', 'label', 't5'],
        num_rows: 1066
    })
})
```

```python
data['test'][0]
```

```
{'text': 'lovingly photographed in the manner of a golden book sprung to life , stuart little 2 manages sweetness largely without stickiness .',
'label': 1,
't5': 'Is the following sentence positive or negative?lovingly photographed in the manner of a golden book sprung to life , stuart little 2 manages sweetness largely without stickiness .'}
```

```python
import numpy as np
from tqdm import tqdm
from transformers.pipelines.pt_utils import KeyDataset

#Run inference
y_pred = []
#tqdm是显示进度条，可视化处理进度
for output in tqdm(pipe(KeyDataset(data["test"],"t5")),total=len(data["test"])):
    text = output[0]["generated_text"]
    y_pred.append(0 if text=="negative"else 1)
```

```
100%|██████████| 1066/1066 [00:41<00:00, 25.52it/s]
```

```python
from sklearn.metrics import classification_report

def evaluate_performance(y_ture, y_pred):
    """Create and print the classification report"""
    performance = classification_report(
            y_ture, y_pred,
            target_names=["Negative Review", "Positive Review"]
    )
    print(performance)
```

```python
evaluate_performance(data["test"]["label"], y_pred)
```

```
                precision    recall  f1-score   support

Negative Review       0.83      0.84      0.83       533
Positive Review       0.84      0.83      0.83       533

      accuracy                           0.83      1066
     macro avg       0.83      0.83      0.83      1066
  weighted avg       0.83      0.83      0.83      1066
```